Stratified RMST Boosting for Subgroup Identification

Augmenting the Value Function with Prognostic Strata

Yue Shentu & Natasha

May 28, 2026

Revised v2: corrected gradient scaling

Contents

1 Introduction

This document defines a stratified version of RMST-based subgroup boosting ("RMST CAV-Boost"). The original method learns a soft subgroup assignment $p_i \in [0, 1]$ (the probability that patient i belongs to the treatment-preferring subgroup) by optimizing a value function through gradient boosting trees. The proposed refinement replaces the original pooled value function with one computed within prognostic strata and aggregated.

There are two distinct ways to use stratification:

1. **Stratified evaluation** (proven, §4): Train p_i using the original CAVBoost, then evaluate subgroup performance via the stratified value function. This approach gave a 64% variance reduction in simulations.

2. **Stratified training** (experimental, §5): Replace the value function used during boosting with the stratified version. The gradient is modified to incorporate stratum-specific RMST differences. This is mathematically sound but sensitive to stratum size.

2 Notation

For each subject $i = 1, \ldots, N$, we observe:

- $\boldsymbol{Z}_i$: p-vector of baseline covariates;
- $A_i \in \{0, 1\}$: treatment (1 = active, 0 = control);
- $U_i = \min(T_i, C_i, \tau)$: observed time;
- $\widetilde{\delta}_i = \mathbf{1}\{\min(T_i, \tau) \leq C_i\}$: event within τ before censoring.

The restricted mean survival time (RMST) at horizon τ is $\mu(\tau) = \int_0^\tau S(t)\, dt$, estimated by the area under the Kaplan–Meier curve.

3 Prognostic Stratification

A prognostic score s_i is a univariate summary of $\boldsymbol{Z}_i$ that predicts survival under the control condition. Because treatment is randomized, we can use all patients (both arms) to estimate s_i as long as treatment is excluded from the predictor set.

Three methods for constructing s_i:

1. **Pseudo-RMST + XGBoost**: Compute jackknife pseudo-observations $\widetilde{\mu}_i$ at τ using the `pseudo` package, then fit XGBoost regression on $\boldsymbol{Z}_i$ to predict $\widetilde{\mu}_i$. The prediction is s_i.

2. **XGBoost survival Cox**: Fit XGBoost with `objective="survival:cox"` on $\boldsymbol{Z}_i$ (not treatment). The linear predictor is s_i.

3. **Standard Cox PH**: Fit Cox($\boldsymbol{Z}_i$) and use the linear predictor as s_i.

Stratify s_i into K groups by its quantiles. Let $\mathcal{S}_k$ be the index set of stratum k, with $n_k = |\mathcal{S}_k|$ patients. For $K = 4$, these are quartiles; $K = 6$ and $K = 8$ are used in sensitivity analyses.

4 Stratified Evaluation

In this mode, p_i is trained using the original (unstratified) CAVBoost, and the stratification is applied only when evaluating the resulting subgroup. This separates the learning problem (which the original CAVBoost handles well) from the evaluation problem (where stratification provides context-dependent insight).

4.1 Within-Stratum RMST Difference

Within each stratum k, compute two RMST differences, mirroring the original CAVBoost:

- $d_k^{(1)}$: RMST difference (treated minus control) estimated via p_i-weighted KM within stratum k;
- $d_k^{(2)}$: RMST difference estimated via $(1 - p_i)$-weighted KM within stratum k.

These depend on p_i through the weights, exactly as in the original.

4.2 Weighted Value Function

Given soft subgroup probabilities p_i, compute for each stratum:

$$W_k = \sum_{i \in \mathcal{S}_k} p_i, \qquad n_k - W_k = \sum_{i \in \mathcal{S}_k} (1 - p_i).$$

The stratified value function (unnormalized) is:

$$V_{\text{strat}} = \sum_{k=1}^{K} W_k \, d_k^{(1)} \qquad - \qquad \sum_{k=1}^{K} (n_k - W_k) \, d_k^{(2)}.$$

The W_k and $n_k - W_k$ terms weight each stratum by the effective size of the treatment-preferring and control-preferring populations, making V_{strat} a soft-weighted aggregate of stratum-level RMST differences.

5 Stratified Training

In this mode, the value function used during XGBoost training is itself stratified. This changes the gradient that each tree fits, potentially altering the learned p_i.

5.1 Value Function

The stratified value function mirrors the original CAVBoost structure exactly, but applied within each stratum. Recall that the original value function is:

$$V_{\text{orig}} = \Big(\sum_i p_i\Big)\widehat{\Delta}^{(1)} - \Big(\sum_i (1 - p_i)\Big)\widehat{\Delta}^{(2)},$$

where $\widehat{\Delta}^{(1)}$ uses p_i-weighted KM (subgroup 1, treatment-preferring) and $\widehat{\Delta}^{(2)}$ uses $(1-p_i)$-weighted KM (subgroup 2, control-preferring).

The stratified version applies the same logic within each stratum k. Define within-stratum subgroup 1 KM (p-weighted) and subgroup 2 KM ((1-p)-weighted), giving two RMST differences per stratum:

$$d_k^{(1)} = \widehat{\mu}_{1,k}^{(1)}(\tau) - \widehat{\mu}_{0,k}^{(1)}(\tau),$$
$$d_k^{(2)} = \widehat{\mu}_{1,k}^{(2)}(\tau) - \widehat{\mu}_{0,k}^{(2)}(\tau),$$

where $\widehat{\mu}_{a,k}^{(1)}(\tau)$ is the p-weighted KM RMST within stratum k, arm a, and $\widehat{\mu}_{a,k}^{(2)}(\tau)$ is the $(1-p)$-weighted equivalent.

The stratified value function is:

$$\boxed{V_{\text{strat}}(\boldsymbol{p}) = \sum_{k=1}^{K} W_k \, d_k^{(1)} \qquad - \qquad \sum_{k=1}^{K} (n_k - W_k) \, d_k^{(2)}}$$

where $W_k = \sum_{i \in \mathcal{S}_k} p_i$ and $n_k = |\mathcal{S}_k|$.

This is the same *difference of deltas* format as the original, but the KM computations are restricted to each stratum. No normalization by N is applied, matching the scale convention of the original.

5.2 Analytic Gradient

The gradient mirrors the original CAVBoost derivation. For patient j in stratum k, taking the derivative of each term in V_{strat} with respect to $\eta_j = \log(p_j/(1-p_j))$:

Term 1: $\dfrac{\partial}{\partial \eta_j}\left[W_k \, d_k^{(1)}\right] = p_j(1-p_j)\, d_k^{(1)} + W_k \, \dfrac{\partial d_k^{(1)}}{\partial \eta_j}$

Term 2: $-\dfrac{\partial}{\partial \eta_j}\left[(n_k - W_k)\, d_k^{(2)}\right] = p_j(1-p_j)\, d_k^{(2)} - (n_k - W_k)\, \dfrac{\partial d_k^{(2)}}{\partial \eta_j}$

The chain rule $\partial d_k^{(g)}/\partial \eta_j = p_j(1-p_j)\, \partial d_k^{(g)}/\partial p_j$ gives the full gradient:

$$\boxed{\frac{\partial V_{\text{strat}}}{\partial \eta_j} = p_j(1-p_j)\Big[d_k^{(1)} + d_k^{(2)} + W_k \, \frac{\partial d_k^{(1)}}{\partial p_j} - (n_k - W_k)\, \frac{\partial d_k^{(2)}}{\partial p_j}\Big].}$$

The XGBoost loss function minimizes, so the gradient passed to the tree learner is $g_j = -\partial V_{\text{strat}}/\partial \eta_j$.

The influence functions. The terms $\partial d_k^{(1)}/\partial p_j$ and $\partial d_k^{(2)}/\partial p_j$ are the derivatives of the within-stratum p-weighted and $(1-p)$-weighted RMST differences with respect to patient j's subgroup probability. They are computed by the same influence-function machinery as the original CAVBoost (the hazard_inc function in the R code), evaluated on the events within stratum k only. The per-stratum computation is K times faster than the full-sample version because each stratum has roughly $n_k \approx N/K$ patients.

Block-diagonal structure. A critical property: $\partial d_k^{(g)}/\partial p_j = 0$ for $j \notin \mathcal{S}_k$, because patient j does not contribute to the KM within stratum k. This means the gradient for patient j depends only on patients in the same stratum. The XGBoost regression tree must therefore learn stratum-relevant

splits from the feature vector $\boldsymbol{Z}_j$, relying on the fact that the prognostic score s_i (used to define strata) is a function of $\boldsymbol{Z}_i$.

Connection to the original. The original CAVBoost gradient (from `rmst_cavboost_clean.R`) is:

$$g_j^{(\text{orig})} = -p_j(1-p_j)\Big[\widehat{\Delta}^{(1)} + \Big(\sum_i p_i\Big)\frac{\partial\widehat{\Delta}^{(1)}}{\partial p_j} + \widehat{\Delta}^{(2)} - \Big(\sum_i (1-p_i)\Big)\frac{\partial\widehat{\Delta}^{(2)}}{\partial p_j}\Big].$$

The stratified version is identical in structure, but with each term replaced by its stratum-level analogue:

$$\begin{aligned}
\widehat{\Delta}^{(1)} &\leftrightarrow d_k^{(1)}, & \widehat{\Delta}^{(2)} &\leftrightarrow d_k^{(2)},\\
\sum_i p_i &\leftrightarrow W_k, & \sum_i (1-p_i) &\leftrightarrow n_k - W_k,\\
\frac{\partial\widehat{\Delta}^{(1)}}{\partial p_j} &\leftrightarrow \frac{\partial d_k^{(1)}}{\partial p_j}, & \frac{\partial\widehat{\Delta}^{(2)}}{\partial p_j} &\leftrightarrow \frac{\partial d_k^{(2)}}{\partial p_j}.
\end{aligned}$$

The implementation is a direct modification of the original: the KM/hazard-increment computation loops over strata instead of the full sample.

5.3 Implementation

The algorithm mirrors the original CAVBoost, but the KM and influence computations loop over strata:

1. Pre-compute prognostic score s_i and stratum assignments $\mathcal{S}_k$. These are fixed.
2. Initialize $\eta_i = 0$ ($p_i = 0.5$).
3. For each boosting round:
   (a) Compute $p_i = 1/(1+e^{-\eta_i})$.
   (b) For each stratum k:
      i. Compute p-weighted KM within $\mathcal{S}_k$, get $d_k^{(1)}$ and influence $\partial d_k^{(1)}/\partial p_j$.
      ii. Compute $(1-p)$-weighted KM within $\mathcal{S}_k$, get $d_k^{(2)}$ and influence $\partial d_k^{(2)}/\partial p_j$.
      iii. Assemble gradient for patients in $\mathcal{S}_k$: $g_j = -p_j(1-p_j)[d_k^{(1)} + d_k^{(2)} + W_k\,\partial d_k^{(1)}/\partial p_j - (n_k - W_k)\,\partial d_k^{(2)}/\partial p_j]$.
   (c) Fit a regression tree to $\{g_j, \boldsymbol{Z}_j\}$.
   (d) Update $\eta_i \leftarrow \eta_i + \eta_{\text{boost}} \times \text{tree}(\boldsymbol{Z}_i)$.
4. After training: $\widehat{G}_i = \mathbf{1}\{p_i > 0.5\}$.

This is a direct adaptation of the original CAVBoost gradient, replacing the global weighted KM with per-stratum weighted KM. The R functions from `rmst_cavboost_clean.R`—in particular the hazard increment and sorted-data machinery—are reused without modification; only the loop structure changes.

6 Simulation Results

All classification metrics are computed on a held-out test set of the same size as the training set. Three methods are compared: original CAVBoost, stratified CAVBoost ($K = 4$, CoxPH prognostic score), and virtual twin (separate Cox PH per arm with RMST prediction).

6.1 Primary DGP

Survival follows a Weibull model with $p = 6$ covariates (Z_1, Z_2 prognostic; Z_3 prognostic and predictive; Z_4–Z_6 noise), $N_{\text{train}} = N_{\text{test}} = 300$, $\tau = 12$, $\approx 30\%$ censoring. Results from 50 Monte Carlo reps:

Method	AUC	Accuracy	FPR	FNR
Original CAVBoost	0.602	0.574	0.492	0.416
Stratified (K=4)	**0.635**	**0.636**	**0.481**	**0.342**
Virtual Twin (Cox PH)	0.705	—	—	—

The stratified version improves AUC by 3.3 points over the original and reduces FNR by 7.4 percentage points, catching more true benefitters. Virtual twin achieves higher AUC but requires fitting separate survival models per arm.

6.2 Pure Predictive (No Prognostic Noise)

Removing all prognostic effects ($\alpha_1 = \alpha_2 = 0$, $z_3^{\text{prog}} = 0$), only Z_3 modifies the treatment effect:

Method	AUC	Accuracy	FPR	FNR
Original CAVBoost	0.620	0.641	0.529	0.326
Stratified (K=4)	**0.656**	**0.654**	**0.452**	**0.326**
Virtual Twin (Cox PH)	0.706	—	—	—

Stratification improves AUC by 3.6 points even without prognostic confounding, showing it also reduces variance from noise covariates.

6.3 Stronger Signal (SubgroupBoost-Style DGP)

With $p = 24$ covariates (1 predictive s_1, 3 prognostic, 20 noise), balanced 50% benefit rate, $N_{\text{train}} = N_{\text{test}} = 600$, adopting the *Zhang et al. (2020)* simulation structure:

Method	AUC	Accuracy	FPR	FNR
Original CAVBoost	0.679	0.713	0.522	0.242
Stratified (K=4)	**0.740**	**0.757**	**0.469**	**0.200**
Virtual Twin (Cox PH)	0.762	—	—	—

The stratification gain is largest here: AUC +6.1 points and FPR -5.3 points. The gap to virtual twin shrinks from 8.3 points (original) to 2.2 points.

6.4 Complex Boundary Scenarios

The *Zhang et al. (2020)* paper defines four additional scenarios with increasing boundary complexity, using 52 covariates (50 noise, 2 predictive S_1, S_2), $N_{\text{train}} = 500$, $N_{\text{test}} = 2000$, and 4 prognostic covariates with effect $\beta_Z = 0.4$ (Setting 2). In these scenarios the treatment effect surface is nonlinear, which exposes the limitations of the linear Cox PH model used by virtual twin.

	Hold-out AUC			
Scenario (boundary)	Original	**Stratified**	VT	Gain
S1: Linear ($S_1 = 0$)	0.798	**0.964**	0.956	+0.166
S3: U-shaped (S_1)	0.918	**0.977**	0.509	+0.059
S4: Enclave ($S_1 \times S_2$)	0.844	0.765	0.506	−0.079
S5: S-shaped (S_1, disjoint)	0.918	0.849	0.596	−0.069

The key findings are:

- **Virtual twin collapses on nonlinear boundaries** (S3: 0.509, S4: 0.506, S5: 0.596). The per-arm linear Cox PH cannot capture non-monotone or interaction-based treatment effect surfaces.
- **Stratified CAVBoost excels when the subgroup boundary is smooth** (S1: +16.6 AUC points, S3: +5.9). The prognostic score stratification aligns with smoothly varying treatment effects.
- **Stratification hurts on disjoint or non-monotone boundaries** (S4 Enclave: −7.9, S5 S-shaped: −6.9). When the true subgroup consists of disconnected regions in covariate space, the monotone CoxPH-based prognostic stratification cannot capture the structure and the gradient is misdirected.
- **Original CAVBoost is robust** across all scenarios (0.80–0.92 AUC), making it a reliable baseline even without stratification.

These results suggest that stratified CAVBoost is most beneficial when the subgroup decision boundary is smooth relative to prognosis, and that adaptive stratification (recomputing strata during boosting) or prognostic scores based on flexible models might extend these gains to more complex boundaries.